

FACULDADE ESAMC UBERLÂNDIA
PROJETO DE EXTENSÃO - 1 SEMESTRE DE 2026

COORDENADOR DE EXTENSÃO: Prof. Yassushi Okada

DISCIPLINA: Redes de Computadores II

CURSOS: Sistemas de Informação

EQUIPE:

Andressa da Silva Santos
Gabriel Pedroso Naves Monteiro
Matheus Alves Bonetti
Pedro Valêncio Souza Marques Moutinho
João Pedro Ferreira Alves

Andressa da Silva Santos
Gabriel Pedroso Naves Monteiro
Pedro Valêncio Souza Marques Moutinho
Matheus Alves Bonetti
João Pedro Ferreira Alves

DOCUMENTOR

Inteligência Artificial aplicada ao auxílio educacional e preparação para o ENADE

Relatório apresentado ao
curso de graduação
em Sistemas de Informação
da ESAMC,
como requisito parcial do
Projeto de Extensão
da disciplina Redes de
Computadores II.

Orientador(a): Prof(a).
Yassushi Okada

Uberlândia - MG
2026

RESUMO

O presente projeto apresenta o desenvolvimento do sistema “DocuMentor”, uma Inteligência Artificial educacional criada com o objetivo de auxiliar estudantes em seus processos de aprendizagem, especialmente na preparação para o Exame Nacional de Desempenho dos Estudantes (ENADE).

O sistema foi desenvolvido para oferecer suporte acadêmico através de funcionalidades como resolução de dúvidas, explicações de conteúdos, utilização de questões anteriores do ENADE, auxílio em estudos personalizados e geração de perguntas relacionadas aos temas estudados pelos usuários.

Além de auxiliar na preparação para avaliações acadêmicas, o DocuMentor também busca democratizar o acesso ao aprendizado, oferecendo uma ferramenta tecnológica acessível para estudantes que possuem dificuldades de estudo, interpretação de conteúdos e organização acadêmica.

O desenvolvimento do projeto possibilitou a aplicação prática de conhecimentos relacionados à programação, inteligência artificial, desenvolvimento de software e experiência do usuário, integrando tecnologia e impacto social em uma única solução.

SUMÁRIO

SUMÁRIO.....	4
1. INTRODUÇÃO.....	5
1.1 INTELIGÊNCIA ARTIFICIAL NA EDUCAÇÃO.....	6
1.2 O ENADE E AS DIFICULDADES DOS ESTUDANTES.....	6
2. OBJETIVOS.....	7
2.1 OBJETIVO GERAL.....	7
2.2 OBJETIVOS ESPECÍFICOS.....	7
3. REFERENCIAL TEÓRICO.....	8
3.1 INTELIGÊNCIA ARTIFICIAL APLICADA AO ENSINO.....	8
3.2 TECNOLOGIAS EDUCACIONAIS.....	8
3.3 APRENDIZAGEM ASSISTIDA POR IA.....	8
4. METODOLOGIA.....	9
4.1 PLANEJAMENTO DO PROJETO.....	9
4.2 DESENVOLVIMENTO DO SISTEMA.....	9
Etapa 1 – Levantamento de Requisitos.....	9
Etapa 2 – Planejamento da Arquitetura.....	9
Etapa 3 – Desenvolvimento do Back-end.....	10
Figura 1 — Estrutura inicial do projeto.....	10
4.3 ESTRUTURA DA APLICAÇÃO.....	10
Figura 2 — Organização do código-fonte.....	11
5. DESENVOLVIMENTO DO PROJETO.....	14
5.1 ESTRUTURA DO SISTEMA.....	14
Figura 3 — Interface inicial do sistema.....	14
5.2 FUNCIONALIDADES DA IA.....	14
Figura 4 — Sistema de perguntas e respostas.....	15
5.3 INTERFACE DO USUÁRIO.....	17
Figura 5 — Desenvolvimento da interface.....	17
5.4 INTEGRAÇÃO DAS FUNCIONALIDADES.....	18
Figura 6 — Integração do sistema.....	18
6. RESULTADOS E DISCUSSÕES.....	18
7. CONCLUSÃO.....	19
8. REFERÊNCIAS BIBLIOGRÁFICAS.....	20

1. INTRODUÇÃO

A tecnologia vem transformando significativamente a forma como estudantes aprendem e acessam informações. Entre essas transformações, destaca-se o avanço da Inteligência Artificial (IA), que passou a ser utilizada em diferentes áreas da sociedade, incluindo a educação.

Ferramentas inteligentes vêm sendo utilizadas para auxiliar estudantes no aprendizado, proporcionando explicações rápidas, conteúdos personalizados e apoio em atividades acadêmicas. Dentro desse contexto, surgiu o projeto “DocuMentor”, desenvolvido com o objetivo de criar uma plataforma de auxílio educacional baseada em Inteligência Artificial.

O sistema possui como principal foco auxiliar estudantes na preparação para o ENADE, utilizando questões anteriores, explicações detalhadas, auxílio em dúvidas e suporte ao aprendizado. Entretanto, a plataforma também pode ser utilizada em outras áreas de estudo, ampliando sua aplicação acadêmica.

Além do desenvolvimento tecnológico, o projeto também possui caráter social, buscando auxiliar estudantes que possuem dificuldades de aprendizagem, acesso limitado a materiais de estudo ou necessidade de reforço acadêmico.

1.1 INTELIGÊNCIA ARTIFICIAL NA EDUCAÇÃO

A Inteligência Artificial aplicada à educação vem crescendo nos últimos anos devido à sua capacidade de personalizar o aprendizado e auxiliar estudantes em tempo real.

Sistemas inteligentes conseguem analisar perguntas, fornecer respostas contextualizadas e adaptar conteúdos conforme a necessidade do usuário. Dessa forma, a IA se torna uma ferramenta importante no apoio educacional, principalmente em ambientes acadêmicos e universitários.

O uso dessas tecnologias também contribui para tornar o estudo mais acessível, dinâmico e interativo, aproximando os estudantes da tecnologia de maneira positiva.

1.2 O ENADE E AS DIFICULDADES DOS ESTUDANTES

O Exame Nacional de Desempenho dos Estudantes (ENADE) é uma avaliação aplicada aos estudantes do ensino superior com o objetivo de medir o desempenho acadêmico dos cursos universitários no Brasil.

Muitos estudantes enfrentam dificuldades durante a preparação para o exame, principalmente relacionadas à interpretação de conteúdos, organização dos estudos e acesso a materiais de apoio.

Diante desse cenário, o DocuMentor foi pensado como uma ferramenta capaz de auxiliar nesse processo, oferecendo questões, explicações e suporte educacional através de Inteligência Artificial.

2. OBJETIVOS

2.1 OBJETIVO GERAL

Desenvolver uma plataforma baseada em Inteligência Artificial capaz de auxiliar estudantes em seus estudos, especialmente na preparação para o ENADE, oferecendo suporte educacional através de perguntas, explicações e conteúdos acadêmicos.

2.2 OBJETIVOS ESPECÍFICOS

- Desenvolver uma interface intuitiva e acessível;
- Criar um sistema capaz de responder dúvidas acadêmicas;
- Auxiliar estudantes na preparação para o ENADE;
- Disponibilizar questões e explicações educativas;
- Incentivar o uso da tecnologia na educação;
- Promover inclusão e apoio acadêmico para estudantes com dificuldades de aprendizagem.

3. REFERENCIAL TEÓRICO

3.1 INTELIGÊNCIA ARTIFICIAL APLICADA AO ENSINO

A Inteligência Artificial consiste em sistemas computacionais capazes de simular comportamentos humanos, como interpretação, aprendizado e resolução de problemas.

Na educação, essas tecnologias podem ser utilizadas para criar ambientes de aprendizagem personalizados, auxiliando estudantes na resolução de dúvidas e no desenvolvimento acadêmico.

3.2 TECNOLOGIAS EDUCACIONAIS

As tecnologias educacionais possuem papel importante na modernização do ensino, permitindo maior acessibilidade e dinamismo nos estudos.

Plataformas digitais, sistemas inteligentes e ferramentas online ampliam o acesso à informação e contribuem para uma aprendizagem mais eficiente.

3.3 APRENDIZAGEM ASSISTIDA POR IA

Sistemas educacionais baseados em IA conseguem adaptar conteúdos conforme as necessidades do usuário, oferecendo explicações mais rápidas e suporte contínuo ao estudante.

Esse modelo de aprendizado vem sendo utilizado como apoio complementar ao ensino tradicional.

4. METODOLOGIA

4.1 PLANEJAMENTO DO PROJETO

O desenvolvimento do projeto iniciou-se através de pesquisas relacionadas à Inteligência Artificial, plataformas educacionais e dificuldades enfrentadas por estudantes universitários durante a preparação para avaliações acadêmicas.

Após os estudos iniciais, foram definidos os objetivos do sistema, suas funcionalidades e a estrutura geral da aplicação.

4.2 DESENVOLVIMENTO DO SISTEMA

O sistema **DocuMentor** foi desenvolvido com o objetivo de criar uma plataforma inteligente capaz de auxiliar estudantes em seus estudos, especialmente na preparação para o ENADE. Para isso, foram utilizadas tecnologias modernas que possibilitam a integração entre Inteligência Artificial e aplicações web.

O **back-end** da aplicação foi desenvolvido em **Python**, linguagem amplamente utilizada em projetos de Inteligência Artificial, ciência de dados e desenvolvimento de software devido à sua flexibilidade, facilidade de manutenção e grande quantidade de bibliotecas disponíveis.

Já o **front-end** foi desenvolvido utilizando o **Streamlit**, uma biblioteca Python voltada para a criação de interfaces web interativas de forma rápida e eficiente. A escolha dessa tecnologia permitiu desenvolver uma interface intuitiva, facilitando a interação entre os usuários e a Inteligência Artificial.

O processo de desenvolvimento foi dividido nas seguintes etapas:

Etapas 1 – Levantamento de Requisitos

Inicialmente, foram realizadas pesquisas sobre as principais dificuldades enfrentadas por estudantes durante a preparação para o ENADE. A partir dessas informações, foram definidas as funcionalidades essenciais da plataforma, como sistema de perguntas e respostas, explicação de conteúdos e suporte aos estudos.

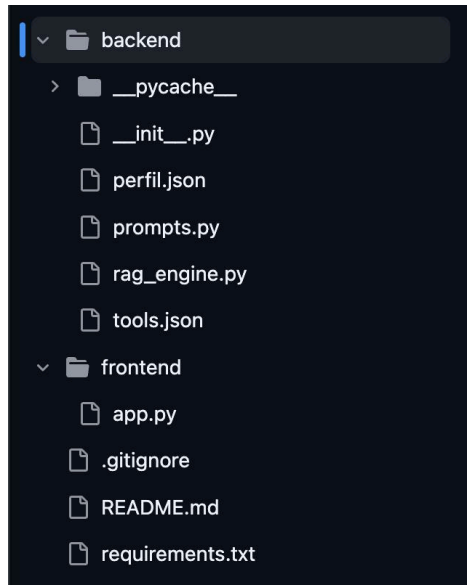
Etapas 2 – Planejamento da Arquitetura

Após a definição dos requisitos, foi elaborada a estrutura do sistema, separando os componentes responsáveis pela interface do usuário, processamento das solicitações e integração com os recursos de Inteligência Artificial.

Etapas 3 – Desenvolvimento do Back-end

Nesta etapa, foi implementada toda a lógica da aplicação utilizando Python. O back-end ficou responsável pelo processamento das perguntas realizadas pelos usuários, gerenciamento dos dados e comunicação com os modelos de Inteligência Artificial utilizados pelo sistema.

Figura 1 — Estrutura inicial do projeto



4.3 ESTRUTURA DA APLICAÇÃO

A aplicação foi organizada de maneira modular, permitindo melhor separação entre as funcionalidades do sistema e facilitando futuras manutenções e melhorias.

O back-end desenvolvido em Python ficou responsável pela lógica principal do sistema, processamento das perguntas realizadas pelos usuários e integração com os recursos de Inteligência Artificial.

Figura 2 — Organização do código-fonte

```
backend > rag.engine.py > ...
1 import json
2 import os
3 import sys
4
5 sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), "..")))
6
7 import re
8 from typing import List
9
10 from dotenv import load_dotenv
11 from groq import Groq
12 from langchain_chroma import Chroma
13 from langchain_community.document_loaders import PyPDFLoader
14 from langchain_huggingface import HuggingFaceEmbeddings
15 from langchain_text_splitters import RecursiveCharacterTextSplitter
16
17 from backend.prompts import prompt_1
18
19 load_dotenv()
20
21 base_dir = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
22 data_dir = os.path.join(base_dir, "data")
23
24
25 class RagEngine:
26     def __init__(self, model_name="sentence-transformers/all-MiniLM-L6-v2"):
27         self.client = Groq(
28             api_key="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"
29         ) # PUT YOUR GROQ API KEY HERE, OR SET IT IN THE .env FILE
30         self.embeddings = HuggingFaceEmbeddings(model_name=model_name)
31         self.vector_store = None
32         self.primm_step = None
33
34     # --- Document Processing ---
35     def clean_text(self, text: str) -> str:
36         emoji_pattern = re.compile(
37             "[
38                 '\U0001f600-\U0001f64f' # emoticons
39                 '\U0001f300-\U0001f5ff' # symbols & pictographs
40                 '\U0001f680-\U0001f6ff' # transport & map symbols
41                 '\U0001f1e0-\U0001f1ff' # flags
42                 '\U00002702-\U000027b0'
43                 '\U000024c2-\U0001f251"
44             ]+",
45             flags=re.UNICODE,
46         )
47
48         return emoji_pattern.sub(r"", text)
```

```
25 class RagEngine:
26
27     def get_chunks(
28         self,
29         document_path: str,
30         chunk_size: float = 1000,
31         chunk_overlap: float = 200,
32         separators: List[str] | None = None,
33     ):
34         # Load the document
35         loader = PyPDFLoader(document_path) # path for the document
36         documents = loader.load()
37
38         for doc in documents:
39             doc.page_content = self.clean_text(doc.page_content)
40
41         if not separators:
42             sample_text = ".join([doc.page_content for doc in documents[:3]])
43             doc_type = self.detect_doc_type(sample_text)
44             separators = self.smart_separators(doc_type=doc_type)
45
46         if not chunk_size:
47             sample_text = ".join([doc.page_content for doc in documents])
48             chunk_size = self.get_adaptative_chunk_size(sample_text)
49             chunk_overlap = chunk_size * 0.13
50
51         # Create the splitter using RecursiveCharacterTextSplitter module, arguments can be changed
52         splitter = RecursiveCharacterTextSplitter(
53             chunk_size=chunk_size,
54             chunk_overlap=chunk_overlap, # Generally 10% - 20% of the chunk size
55             separators=separators,
56             length_function=len,
57         )
58
59         # Generate and return the chunks
60         chunks = splitter.split_documents(documents)
61         print(f"Documents split into {len(chunks)} chunks.")
62         print("Enriching metadata.")
63         chunks = self.enrich_chunks_metadata(chunks)
64         print("Metadata enriched.")
65
66         return chunks
67
68     def detect_doc_type(self, text_sample: str) -> str:
69         if "'''''' in text_sample and ("def" in text_sample or "function" in text_sample):
70             return "code_docs"
71
72         elif any(method in text_sample for method in ["POST", "GET", "PUT", "DELETE"]):
```

```

25 class RagEngine:
91     def detect_doc_type(self, text_sample: str) -> str:
96         return "api_docs"
97
98         return "general_docs"
99
100     def smart_separators(self, doc_type: str = "general") -> list[str]:
101         separators_map = {
102             "code_docs": [
103                 "\n## ", # Headers markdown
104                 "\n### ",
105                 "\n ` ` ", # Blocos de código
106                 "\n\n", # Parágrafos
107                 "\n",
108                 ". ",
109             ],
110             "api_docs": [
111                 "\n# ", # Títulos principais
112                 "\n## ", # Endpoints/secões
113                 "\nPOST ", # Métodos HTTP
114                 "\nGET ",
115                 "\n\n",
116                 "\n",
117             ],
118             "general": [
119                 "\n\n",
120                 "\n",
121                 ". ",
122                 "! ",
123                 "? ",
124             ],
125         }
126         return separators_map.get(doc_type, separators_map["general"])
127
128     def get_adaptative_chunk_size(self, text: str) -> int:
129         if text.count("````") > 5:
130             return 800
131
132         avg_sentence_len = len(text) / (text.count(".") + 1)
133         if avg_sentence_len > 50:
134             return 1200
135
136         return 1000
137
138     def enrich_chunks_metadata(self, chunks):
139         for i, chunk in enumerate(chunks):
140             chunk.metadata["id"] = i
141             chunk.metadata["chunk_size"] = len(chunk.page_content)

```

```

25 class RagEngine:
138     def enrich_chunks_metadata(self, chunks):
143         chunk.metadata["has_code"] = "````" in chunk.page_content
144
145         pedagogical_words = [
146             "conceito",
147             "concept",
148             "entender",
149             "understand",
150             "funciona",
151             "works",
152             "exemplo",
153             "example",
154             "porque",
155             "why",
156         ]
157         chunk.metadata["is_conceptual"] = any(
158             word in chunk.page_content.lower() for word in pedagogical_words
159         )
160
161         lines = chunk.page_content.split("\n")
162         for line in lines[3:]:
163             if line.startswith("#"):
164                 chunk.metadata["section"] = line.strip("# ")
165                 break
166
167         return chunks
168
169     # --- Document Retrieval ---
170     def retrieve_relevant_chunks(self, query, vector_store, k=5):
171         chunks = vector_store.similarity_search(query, k=k)
172         return chunks
173
174     # --- Generation ---
175     def build_message(
176         self, system_prompt, context, user_query, chat_history, prim_step
177     ) -> list[dict[str, None]]:
178
179         # Detecta a língua e reforça no system prompt
180         system_with_lang = (
181             system_prompt
182             + f"\n\nIMPORTANT: The user's message is in the language you must reply in. Do NOT switch languages under any circumstance."
183         )
184
185         message = [{"role": "system", "content": system_with_lang}]
186
187         if chat_history:
188             message.extend(chat_history)

```

```

275 class RagEngine:
276     def build_message(
277         # Wrapper neutro, sem forçar idioma
278         final_prompt = (
279             f"PRIMM Step: {primm_step}\n"
280             f"Context:\n{context}\n\n"
281             f"{user_query}" # + Mensagem do usuário pura, sem wrapper em PT
282         )
283
284         message.append({"role": "user", "content": final_prompt})
285
286         return message
287
288     def format_context_llm(self, chunks=None, user_lvl: str | None = None) -> str:
289         string = f"User level: {user_lvl}\n"
290
291         if chunks:
292             for chunk in chunks:
293                 string += f"Chunk metadata: {chunk.metadata}\nChunk page_content: {chunk.page_content}\n"
294
295         return string
296
297     def call_llm(self, messages) -> str | None:
298         with open("backend/tools.json", "r") as file:
299             tools = json.load(file)
300
301         print(self.primm_step)
302
303         response = self.client.chat.completions.create(
304             messages=messages,
305             model="llama-3.3-70b-versatile",
306             tools=tools,
307             tool_choice="auto",
308         )
309
310         return response.choices[0].message.content
311
312     def generate_message(
313         self, chat_history, user_query, user_level, primm_step
314     ) -> str | None:
315         context = self.format_context_llm(user_lvl=user_level)
316         messages = self.build_message(
317             system_prompt=prompt_1,
318             context=context,
319             chat_history=chat_history,
320             user_query=user_query,
321             primm_step=self.primm_step,
322         )

```

```

25 class RagEngine:
225     def generate_message(
226         response = self.call_llm(messages=messages)
227
228         return response
229
230     # --- Vector Store ---
231     def get_vector_store(self, doc_name, store_name):
232         doc_path = os.path.join(data_dir, "documents", doc_name)
233         chunks = self.get_chunks(doc_path)
234
235         persistent_dir = os.path.join(data_dir, "vectorstore", store_name)
236
237         if not os.path.exists(persistent_dir):
238             print(f"\n--- Creating vector store {store_name} ---")
239             vector_store = Chroma.from_documents(
240                 documents=chunks,
241                 embedding=self.embeddings,
242                 persist_directory=persistent_dir,
243             )
244             print(f"\n--- Finished creating vector store {store_name} ---")
245
246         else:
247             vector_store = Chroma(
248                 persist_directory=persistent_dir, embedding_function=self.embeddings
249             )
250             print(
251                 f"\n--- Vector store {store_name} already exists. No need to initialize ---"
252             )
253
254         self.vector_store = vector_store
255
256     # --- Complete Pipeline ---
257     def main_method(self, user_query) -> str | None:
258         vector_store = self.get_vector_store(doc_name="eloz.pdf", store_name="eloz-api")
259
260         chunks = self.retrieve_relevant_chunks(
261             query=user_query, vector_store=vector_store
262         )
263         formatted_context = self.format_context_llm(chunks, "beginner")
264         message = self.build_message(
265             system_prompt=prompt_1,
266             context=formatted_context,
267             user_query=user_query,
268             chat_history=[],
269             primm_step="0",
270         )
271         response = self.call_llm(messages=message)

```

```

25 class RagEngine:
267     def main_method(self, user_query) -> str | None:
268
269         return response
270
271     # --- Tools ---
272     def update_primm_step(self, primm_step: str):
273         self.primm_step = primm_step
274

```

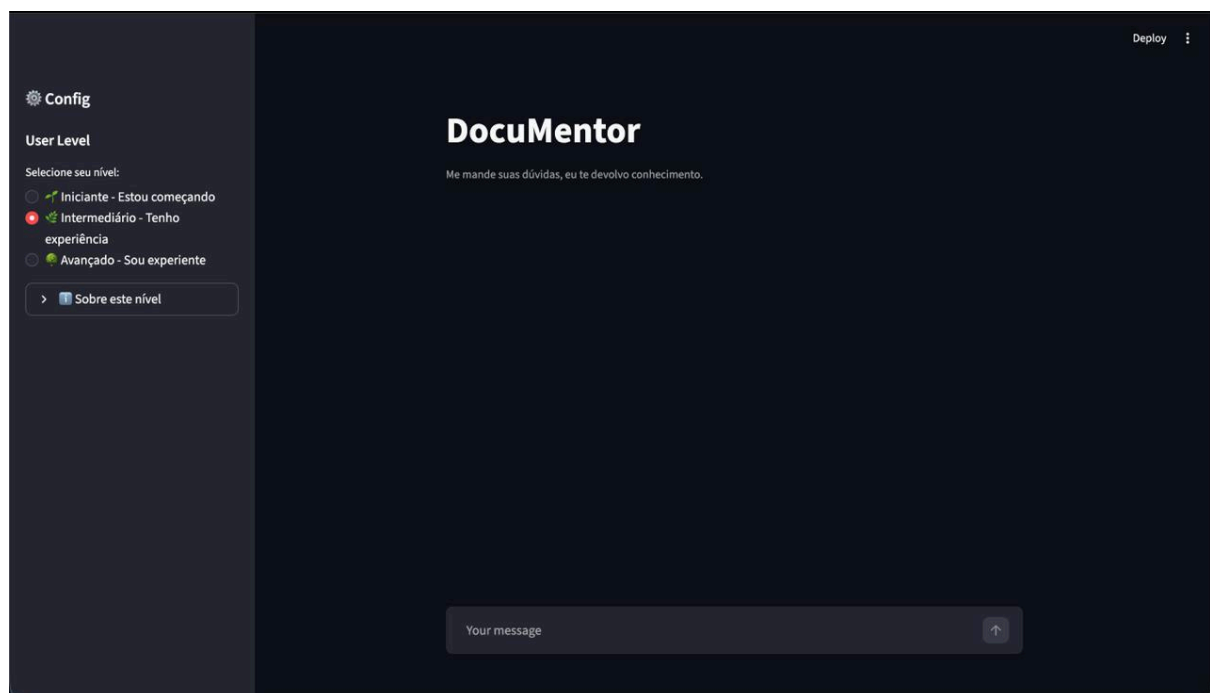
5. DESENVOLVIMENTO DO PROJETO

5.1 ESTRUTURA DO SISTEMA

O DocuMentor foi desenvolvido com foco em simplicidade e acessibilidade, permitindo que estudantes consigam utilizar a plataforma de maneira intuitiva.

O sistema conta com uma interface de interação entre usuário e Inteligência Artificial, possibilitando o envio de perguntas e recebimento de respostas relacionadas aos conteúdos acadêmicos.

Figura 3 — Interface inicial do sistema

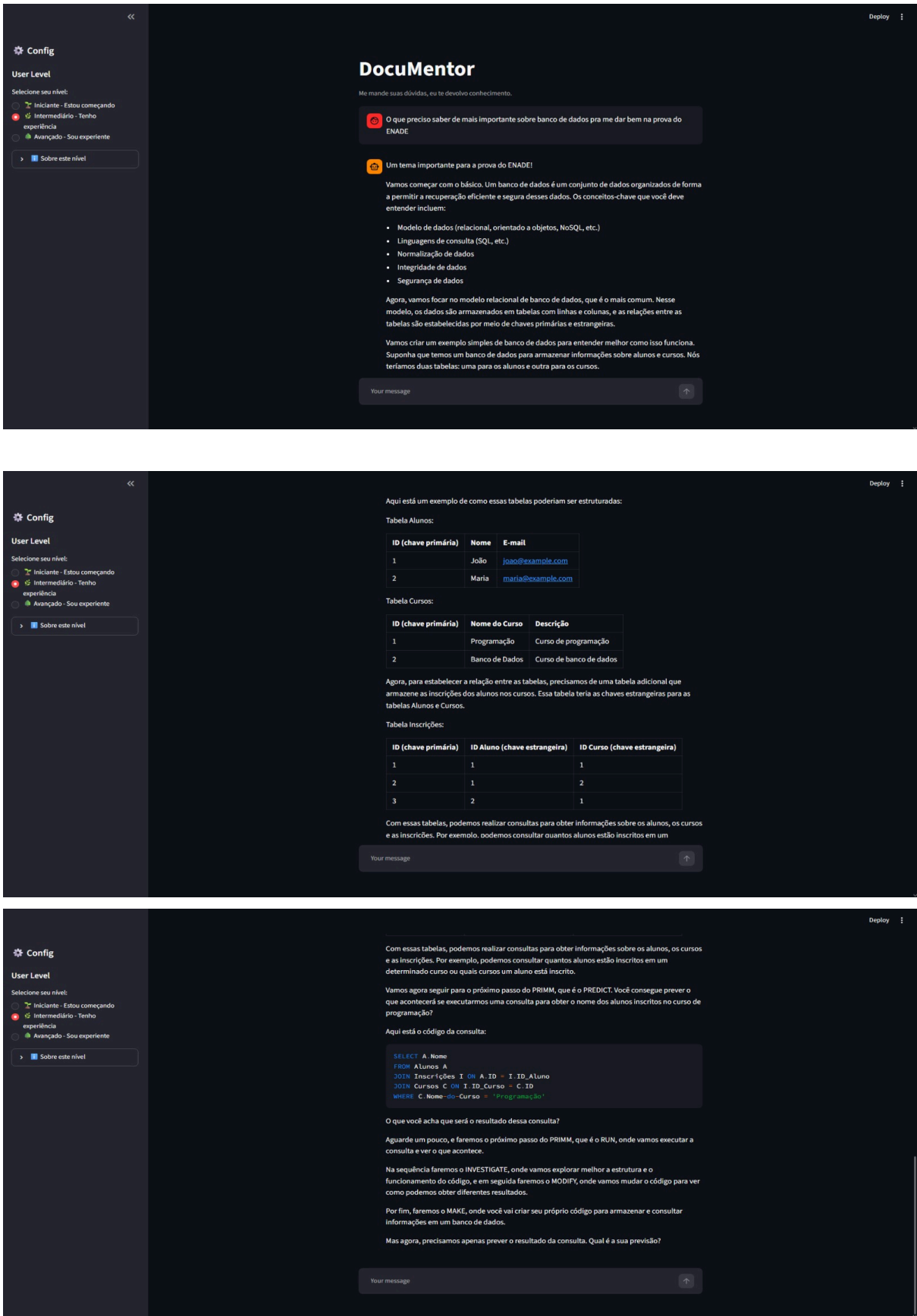


5.2 FUNCIONALIDADES DA IA

Entre as principais funcionalidades implementadas no sistema estão:

- auxílio em dúvidas acadêmicas;
- explicações de conteúdos;
- suporte aos estudos para o ENADE;
- perguntas educativas;
- utilização de questões anteriores;
- interação com Inteligência Artificial.

Figura 4 — Sistema de perguntas e respostas



Config

User Level

Selecione seu nível:

Iniciante - Estou começando

Intermediário - Tenho experiência

Avançado - Sou experiente

Sobre este nível

E sobre infraestrutura e redes

Infraestrutura e redes! Um tema fundamental para entender como os sistemas de informação funcionam.

Vamos começar com o básico. Infraestrutura de TI se refere ao conjunto de recursos físicos e lógicos necessários para suportar os sistemas de informação de uma organização. Isso inclui:

- Hardware (servidores, armazenamento, rede, etc.)
- Software (sistemas operacionais, aplicativos, etc.)
- Redes (LAN, WAN, Wi-Fi, etc.)

As redes de computadores são um componente crítico da infraestrutura de TI. Elas permitem que os dispositivos se comuniquem entre si e compartilhem recursos. As redes podem ser classificadas em diferentes tipos, incluindo:

- LAN (Local Area Network): rede local que conecta dispositivos em uma área geográfica limitada.
- WAN (Wide Area Network): rede que conecta dispositivos em uma área geográfica mais ampla, como uma cidade ou um país.
- Wi-Fi: tecnologia de rede sem fio que permite que os dispositivos se conectem à Internet ou a outras redes sem a necessidade de cabos.

Agora, vamos falar sobre os protocolos de rede. Protocolos de rede são regras que definem como os dispositivos se comunicam uns com os outros. Alguns protocolos de rede comuns incluem:

- TCP/IP (Transmission Control Protocol/Internet Protocol): um conjunto de protocolos que define como os dispositivos se comunicam na Internet.
- HTTP (Hypertext Transfer Protocol): um protocolo que define como os dispositivos se comunicam na Web.

Your message

Deploy

Config

User Level

Selecione seu nível:

Iniciante - Estou começando

Intermediário - Tenho experiência

Avançado - Sou experiente

Sobre este nível

como uma cidade ou um país.

- Wi-Fi: tecnologia de rede sem fio que permite que os dispositivos se conectem à Internet ou a outras redes sem a necessidade de cabos.

Agora, vamos falar sobre os protocolos de rede. Protocolos de rede são regras que definem como os dispositivos se comunicam uns com os outros. Alguns protocolos de rede comuns incluem:

- TCP/IP (Transmission Control Protocol/Internet Protocol): um conjunto de protocolos que define como os dispositivos se comunicam na Internet.
- HTTP (Hypertext Transfer Protocol): um protocolo que define como os dispositivos se comunicam na Web.
- FTP (File Transfer Protocol): um protocolo que define como os dispositivos transferem arquivos uns para os outros.

Aqui está um exemplo de como os protocolos de rede funcionam:

- Quando você digita um endereço de website no seu navegador, o seu computador envia uma solicitação para o servidor da Web usando o protocolo HTTP.
- O servidor da Web recebe a solicitação e responde com as informações solicitadas, também usando o protocolo HTTP.
- O seu computador recebe as informações e as exibe no seu navegador.

Agora que você entendeu melhor como as redes e os protocolos de rede funcionam, vamos seguir para o próximo passo do PRIMM, que é o PREDICT. Você consegue prever o que acontecerá se um dispositivo de rede não estiver configurado corretamente?

Vamos considerar um exemplo. Suponha que você está configurando uma rede LAN em uma empresa e você não configurou o servidor DNS corretamente. O que você acha que acontecerá quando os usuários tentarem acessar a Internet?

Your message

Deploy

5.3 INTERFACE DO USUÁRIO

A interface do sistema foi construída utilizando Streamlit, framework que possibilita o desenvolvimento rápido de aplicações web interativas utilizando Python.

O objetivo foi criar um ambiente simples, organizado e intuitivo, permitindo que estudantes consigam utilizar a plataforma de maneira prática durante seus estudos.

Figura 5 — Desenvolvimento da interface

```
frontend > app.py > ...
1 import os
2 import sys
3
4 sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), "..")))
5
6 import streamlit as st
7
8 from backend.rag_engine import RagEngine
9
10 # Page Configuration
11 st.set_page_config(page_title="DocuMentor", page_icon="📖")
12 st.title("DocuMentor")
13 st.caption("Me mande suas dúvidas, eu te devolvo conhecimento.")
14
15
16 # RAG Initialization
17 @st.cache_resource
18 def init_rag():
19     rag = RagEngine()
20     return rag
21
22
23 rag = init_rag()
24
25 # Creating chat history
26 if "messages" not in st.session_state:
27     st.session_state.messages = []
28     st.session_state.prim_step = 0
29
30 if "user_level" not in st.session_state:
31     st.session_state.user_level = "intermediate"
32
33 if "level_changed" not in st.session_state:
34     st.session_state.level_changed = False
35
36 with st.sidebar:
37     st.header("⚙️ Config")
38
39     st.subheader("User Level")
40     level = st.radio(
41         "Selecione seu nível:",
42         options=["beginner", "intermediate", "advanced"],
43         format_func=lambda x: {
44             "beginner": "👶 Iniciante - Estou começando",
45             "intermediate": "🧑 Intermediário - Tenho experiência",
46             "advanced": "🦾 Avançado - Sou experiente",
47         }[x],
48         index=["beginner", "intermediate", "advanced"].index(
51
52     )
53     key="level_selector",
54
55 # Detectar mudança de nível
56 if level != st.session_state.user_level:
57     st.session_state.user_level = level
58     st.session_state.level_changed = True
59     st.success(f"Nível alterado para: {level}")
60
61 # Descrição do nível selecionado
62 level_descriptions = {
63     "beginner": """
64     **👶 Modo Iniciante:**
65     - Explicações detalhadas com analogias
66     - Conceitos fundamentais sempre explicados
67     - Código com comentários linha a linha
68     - Mais encorajamento e contexto
69     """,
70     "intermediate": """
71     **🧑 Modo Intermediário:**
72     - Equilíbrio entre conceito e implementação
73     - Assume conhecimento básico
74     - Código com comentários principais
75     - Foco em boas práticas
76     """,
77     "advanced": """
78     **🦾 Modo Avançado:**
79     - Discussões técnicas profundas
80     - Menos contexto básico
81     - Edge cases e otimizações
82     - Performance e arquitetura
83     """,
84 }
85
86 with st.expander("📖 Sobre este nível"):
87     st.markdown(level_descriptions[level])
88
89 # Showing chat history
90 for msg in st.session_state.messages:
91     with st.chat_message(msg["role"]):
92         st.write(msg["content"])
93
94 if prompt := st.chat_input():
95     st.session_state.messages.append({"role": "user", "content": prompt})
96
97     with st.chat_message("user"):
98         st.write(prompt)
```

```

93
94 if prompt := st.chat_input():
95     st.session_state.messages.append({"role": "user", "content": prompt})
96
97     with st.chat_message("user"):
98         st.write(prompt)
99
100     with st.chat_message("assistant"):
101         with st.spinner("Pensando..."):
102             response = rag.generate_message(
103                 user_query=prompt,
104                 chat_history=st.session_state.messages[:-1],
105                 user_level=st.session_state.user_level,
106                 primm_step=st.session_state.primm_step,
107             )
108
109         st.write(response)
110         st.session_state.primm_step += 1
111
112     st.session_state.messages.append({"role": "assistant", "content": response})
113

```

5.4 INTEGRAÇÃO DAS FUNCIONALIDADES

Após o desenvolvimento individual das funcionalidades, foram realizados testes para verificar o funcionamento correto do sistema e a comunicação entre os componentes da aplicação.

Figura 6 — Integração do sistema

```

o joaopedro@MacBook-Air-de-Joao DocuMentor % streamlit run frontend/app.py
2026-05-21 19:42:02.145 Uvicorn server started on 0.0.0.0:8501

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://10.23.2.73:8501

```

6. RESULTADOS E DISCUSSÕES

O desenvolvimento do projeto permitiu aplicar conhecimentos relacionados à programação, desenvolvimento de software e Inteligência Artificial em uma solução com foco educacional.

O sistema apresentou resultados positivos durante os testes realizados, demonstrando capacidade de auxiliar estudantes através de respostas rápidas, explicações e apoio aos estudos.

Além do aspecto tecnológico, o projeto também apresentou relevância social, considerando que muitos estudantes possuem dificuldades de aprendizagem e acesso limitado a ferramentas educacionais de qualidade.

O DocuMentor demonstra como a tecnologia pode ser utilizada como ferramenta de inclusão acadêmica e apoio ao desenvolvimento estudantil.

7. CONCLUSÃO

O desenvolvimento do projeto “DocuMentor” possibilitou a criação de uma ferramenta tecnológica voltada ao auxílio educacional, utilizando Inteligência Artificial como suporte aos estudantes durante seus processos de aprendizagem.

A plataforma foi desenvolvida utilizando Python no back-end e Streamlit no front-end, permitindo a construção de uma aplicação funcional, acessível e intuitiva. A integração dessas tecnologias possibilitou desenvolver um sistema capaz de responder dúvidas, auxiliar nos estudos e oferecer suporte acadêmico de maneira prática e interativa.

O principal foco do projeto foi auxiliar estudantes na preparação para o ENADE, utilizando perguntas, explicações e apoio educacional através da Inteligência Artificial. Entretanto, a plataforma também pode ser aplicada em diferentes áreas do aprendizado acadêmico.

Além da experiência técnica adquirida durante o desenvolvimento do sistema, o projeto também proporcionou uma reflexão sobre o impacto social da tecnologia na educação, demonstrando como ferramentas inteligentes podem auxiliar estudantes com dificuldades de aprendizagem e ampliar o acesso ao conhecimento.

Dessa forma, conclui-se que o projeto atingiu seus objetivos, integrando tecnologia, educação e impacto social em uma única solução, além de demonstrar na prática a aplicação de Python e Streamlit no desenvolvimento de sistemas educacionais baseados em Inteligência Artificial.

8. REFERÊNCIAS BIBLIOGRÁFICAS

INEP. Exame Nacional de Desempenho dos Estudantes (ENADE). Brasília: Instituto Nacional de Estudos e Pesquisas Educacionais Anísio Teixeira, 2026. Disponível em: <https://www.gov.br/inep>. Acesso em: 29 maio 2026.

RUSSELL, Stuart; NORVIG, Peter. Inteligência Artificial: uma abordagem moderna. 4. ed. São Paulo: Pearson, 2022.

LUGER, George F. Inteligência Artificial: estruturas e estratégias para a solução de problemas complexos. 6. ed. Porto Alegre: Bookman, 2013.

VALENTE, José Armando. Computadores e conhecimento: repensando a educação. Campinas: UNICAMP/NIED, 1993.

MORAN, José Manuel; MASETTO, Marcos T.; BEHRENS, Marilda Aparecida. Novas tecnologias e mediação pedagógica. 21. ed. Campinas: Papirus, 2018.

PYTHON SOFTWARE FOUNDATION. Python Documentation. Disponível em: Python Documentation. Acesso em: 29 maio 2026.

STREAMLIT INC. Streamlit Documentation. Disponível em: Streamlit Documentation. Acesso em: 29 maio 2026.

GOODFELLOW, Ian; BENGIO, Yoshua; COURVILLE, Aaron. Deep Learning. Cambridge: MIT Press, 2016. Disponível em: Deep Learning Book. Acesso em: 29 maio 2026.

SILVA, Marco. Educação Online. 6. ed. São Paulo: Loyola, 2015.

GITHUB. Repositório oficial do projeto DocuMentor. Disponível em: <https://github.com/Vlencio/DocuMentor/tree/trabalho>